

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 01 -

FUNDAMENTOS E ARQUITETURA DE PIPELINES DE CI/CD

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	15
O QUE VOCÊ VIU NESTA AULA?	17
REFERÊNCIAS.....	18

O QUE VEM POR AÍ?

Na era da nuvem, entregar software de forma rápida e segura é fundamental. Assim, a filosofia DevOps e a automação de processos surgiram como pilares para a inovação. Esta aula foi pensada para você aprofundar seu conhecimento na arquitetura e fundamentos dos pipelines de Integração Contínua (CI) e Implantação Contínua (CD), com foco em ferramentas de ponta. Nosso objetivo é equipar você com a base necessária para liderar a transformação da entrega de software em qualquer organização.

HANDS ON

Preparado(a) para colocar a mão na massa? As videoaulas te guiarão na construção do seu primeiro pipeline de CI usando o GitHub Actions, uma das ferramentas mais relevantes do mercado. Você vai traduzir a teoria do Pipeline as Code (PaC) em um exemplo funcional, automatizando etapas cruciais como o build e os testes de uma aplicação. Usando arquivos YAML, você vai entender como definir e executar um workflow de forma simples e visual. Acompanhe as videoaulas e, sempre que precisar, revise este material de apoio!

SAIBA MAIS

A Essência do CI/CD e a Cultura DevOps

Os pipelines de Integração e Implantação Contínuas (CI/CD) são uma série de etapas definidas que especialistas em desenvolvimento precisam seguir para disponibilizar uma nova versão de um software. A essência desses pipelines está na automação, que elimina tarefas manuais repetitivas, reduzindo erros e acelerando significativamente a entrega de software. A automação de testes, por exemplo, permite que vulnerabilidades e bugs sejam identificados no início do ciclo de desenvolvimento, economizando tempo e esforço no futuro.

A relação entre CI/CD e a cultura DevOps é intrínseca. A metodologia DevOps, que incentiva a colaboração entre as equipes de desenvolvimento e de operações, é materializada nos pipelines de CI/CD. O(a) profissional de DevOps atua como a ponte entre essas áreas, implementando e mantendo os pipelines para garantir que as aplicações sejam desenvolvidas, testadas e implantadas de forma eficiente e sem falhas. Essa abordagem fomenta a cultura de "segurança à esquerda" (shift left security) em que as verificações de segurança são incorporadas desde o início do processo, e não apenas no final, permitindo um feedback rápido sobre possíveis vulnerabilidades.

Esse movimento para a esquerda no ciclo de vida do desenvolvimento não se restringe apenas à segurança, mas também à qualidade. A automação das verificações de código e dos testes unitários faz parte do processo de Integração Contínua, assegurando que o código-fonte quebre o menos possível e que os problemas sejam identificados o mais cedo possível, reduzindo o custo de correção e o tempo de rollback.

O acrônimo CI/CD engloba duas práticas distintas, mas complementares:

- **Integração Contínua (CI):** uma prática de desenvolvimento em que desenvolvedores(as) integram mudanças de código a um repositório compartilhado com frequência, idealmente várias vezes ao dia. Cada integração é verificada por um build automatizado e um conjunto de testes, permitindo a detecção precoce de problemas de integração. A ferramenta

de CI automatiza a verificação da qualidade do código, tornando o desenvolvimento mais eficiente.

- **Entrega Contínua (Continuous Delivery - CD):** uma prática que garante que o software esteja sempre em um estado pronto para ser implantado em produção a qualquer momento, após passar por todas as etapas de teste e validação. A implantação final pode, no entanto, ser um passo manual, permitindo um ponto de controle humano.
- **Implantação Contínua (Continuous Deployment - CD):** a extensão da Entrega Contínua, em que o software validado é automaticamente implantado em produção, sem a necessidade de intervenção humana.

A escolha entre Entrega Contínua e Implantação Contínua reflete o nível de maturidade e tolerância a riscos de uma organização. Muitas empresas de setores regulamentados, como finanças e saúde, optam pela Entrega Contínua para manter um ponto de controle humano, garantindo a conformidade e mitigando riscos, enquanto empresas de tecnologia nativas da nuvem, como a Netflix, buscam a Implantação Contínua para maximizar a agilidade.

A Engenharia por Trás de um Pipeline de CI/CD

Um pipeline de CI/CD é uma estrutura de automação que guia o processo de desenvolvimento de software por uma série de etapas de build, teste e implantação do código. Cada estágio do pipeline é uma unidade lógica no processo de entrega, agindo como um "portão" (gate) que valida um aspecto específico do código. O princípio é que a qualidade do código aumenta à medida que ele avança no pipeline, pois mais verificações são realizadas. Se um problema é detectado em um estágio inicial, o código é impedido de prosseguir para os estágios seguintes e o feedback é enviado imediatamente à equipe para correção.

Os pipelines podem ser iniciados por uma variedade de gatilhos (triggers), como um push de código para o repositório, a abertura de um pull request ou mesmo um agendamento (schedule). Uma vez disparado, o pipeline passa pelos seguintes estágios principais:

- **Build:** neste estágio, o código-fonte mais recente é obtido do repositório, compilado e empacotado em um artefato implantável (por exemplo, um arquivo .jar, .war ou uma imagem Docker). Testes unitários e análises de

código estático (linters) também são executados para verificar a qualidade do código.

- **Test:** se o estágio de build for bem-sucedido, o artefato é submetido a testes mais complexos, como testes de integração, testes de desempenho e testes de segurança. Este estágio é uma porta de entrada (gate) que garante a qualidade do software antes da implantação.
- **Deploy:** o artefato validado no estágio de teste é implantado em um ambiente, que pode ser de homologação, staging ou produção.

A lógica e a execução do pipeline são orquestradas por componentes arquiteturais:

- **Workflows:** o workflow é o processo automatizado que será executado. Ele é configurado por um arquivo YAML e pode conter um ou mais jobs.
- **Jobs:** um job é um conjunto de steps que é executado em um mesmo runner (a máquina virtual que executa a tarefa). Por padrão, os jobs são executados em paralelo, mas é possível definir dependências entre eles para garantir uma sequência lógica.
- **Steps:** cada step é uma tarefa individual dentro de um job. Um step pode executar um comando de shell (run) ou chamar uma action (uses).
- **Actions:** uma action é um conjunto de código pré-definido e reutilizável que executa uma tarefa específica. O uso de actions reduz a quantidade de código repetitivo e democratiza a automação de tarefas complexas.

Pipeline as Code (PaC): A Base da Automação Moderna

O conceito de Pipeline as Code (PaC) trata a definição dos pipelines como código-fonte, armazenando-os em arquivos de configuração, como os arquivos YAML do GitHub Actions. Essa abordagem é fundamental para a automação moderna, pois traz os princípios da engenharia de software para o processo de entrega. O PaC permite o versionamento, a auditabilidade e a colaboração no desenvolvimento dos pipelines, garantindo que a forma de entrega evolua junto com o código da aplicação. Isso elimina a dependência de interfaces gráficas e processos manuais, promovendo padronização e escalabilidade para as equipes.

O Jenkins, por exemplo, oferece duas sintaxes para a criação de pipelines as code: a Declarativa e a Scripted. Embora a sintaxe Scripted seja mais flexível e programável, a sintaxe Declarativa é mais recente, mais fácil de ler e mais simples para a pessoa iniciante, sendo a abordagem recomendada pela maioria. Já o GitHub Actions oferece o conceito de reusable workflows e composite actions para a reutilização de código.

Ambos ajudam a evitar a duplicação de conteúdo, mas possuem diferenças importantes: reusable workflows permitem a reutilização de um workflow inteiro, com múltiplos jobs e steps, enquanto composite actions combinam múltiplos steps que podem ser executados dentro de um job. Reusable workflows podem, por exemplo, usar segredos do repositório, enquanto composite actions não têm acesso a eles.

O Papel do DevSecOps e a Segurança “à Esquerda”

A segurança no pipeline de CI/CD evoluiu de uma fase tardia e reativa para uma prática proativa e contínua, uma filosofia conhecida como **DevSecOps** ou segurança “à esquerda” (shift left security). Em vez de esperar que o software esteja pronto para testar sua segurança, o DevSecOps integra ferramentas e práticas de segurança desde as fases iniciais de planejamento e codificação.

As ferramentas de segurança em um pipeline DevSecOps podem ser integradas em diversos pontos:

- **Análise de Código Estático (SAST):** ferramentas de SAST analisam o código-fonte em busca de vulnerabilidades antes mesmo de ser compilado. Elas funcionam como uma "caixa branca", verificando o código estático. Essa abordagem permite a correção de problemas de segurança diretamente no código, no início do ciclo, com um feedback imediato.
- **Análise de Dependências (SCA):** a maioria das aplicações modernas utiliza bibliotecas e pacotes de terceiros. As ferramentas de SCA identificam vulnerabilidades conhecidas nessas dependências, garantindo que o software não incorpore riscos de segurança de fora.
- **Análise de Código Dinâmico (DAST):** diferente do SAST, as ferramentas de DAST analisam a aplicação em tempo de execução, simulando ataques para identificar vulnerabilidades.

- **Automação de Políticas:** o DevSecOps depende da automação para aplicar políticas de segurança de forma consistente. Ferramentas de configuração como Ansible e Puppet podem ser usadas para gerenciar e impor políticas, garantindo que os ambientes de produção e staging sigam os padrões de conformidade.

A adoção do DevSecOps promove a colaboração entre as equipes de desenvolvimento, segurança e operações. O objetivo não é apenas encontrar e corrigir vulnerabilidades, mas também educar sobre práticas de codificação segura, transformando a segurança de um obstáculo para um motor de inovação.

Ferramentas e Plataformas de Automação

A escolha da ferramenta de automação de CI/CD é uma decisão estratégica que depende das necessidades da organização. Duas das plataformas mais influentes no cenário são o Jenkins e o GitHub Actions, cada uma com características e casos de uso distintos.

O **Jenkins** é um servidor de automação open-source com mais de uma década de história, amplamente utilizado para orquestrar builds, testes e deploys. Sua principal vantagem é a flexibilidade incomparável, sustentada por um ecossistema de mais de 1.800 plugins que permitem a integração com praticamente qualquer ferramenta no mercado. Como uma solução auto-hospedada (self-hosted), o Jenkins oferece controle total sobre o ambiente e a segurança, sendo ideal para empresas com infraestruturas legadas, ambientes air-gapped ou requisitos de conformidade rigorosos. No entanto, essa flexibilidade exige uma equipe dedicada para gerenciar a infraestrutura, a manutenção e a segurança da instância.

Uma das arquiteturas de maior sucesso do Jenkins é o modelo Master/Agent. O **Jenkins Master** é o servidor central que gerencia o sistema, agendando jobs e monitorando os **Jenkins Agents**. O **Jenkins Agent** é a máquina ou contêiner que executa o job em nome do master. Essa arquitetura permite que as equipes distribuam a carga de trabalho, executem tarefas em ambientes isolados com diferentes configurações e escalem a capacidade de build horizontalmente, adicionando mais agentes conforme a necessidade. A comunicação entre o master e o agent é feita de forma segura dentro da rede interna, o que contribui para a segurança e a tolerância a falhas.

Por outro lado, o **GitHub Actions** é uma plataforma moderna e nativa do GitHub, que permite a automação de workflows diretamente no repositório de código. Sua facilidade de uso, com configuração zero para fluxos básicos e workflows definidos em YAML, o torna ideal para equipes menores e projetos de código aberto. O vasto e crescente Marketplace de Actions democratiza a automação, atuando como um ecossistema de blocos de construção reutilizáveis para tarefas complexas, como autenticação em provedores de nuvem e publicação de pacotes. A natureza de Software as a Service (SaaS) do GitHub Actions simplifica a jornada de CI/CD, eliminando a sobrecarga de gerenciamento de infraestrutura, embora com limites de uso e um modelo de custo baseado em consumo para repositórios privados.

A escolha entre essas ferramentas se resume a um dilema de arquitetura: **controle vs. conveniência**. O Jenkins se mantém relevante para empresas que precisam de controle granular e personalização para lidar com infraestruturas complexas. Já o GitHub Actions, com sua abordagem SaaS, simplifica a jornada e acelera a inovação para equipes que operam em ambientes nativos da nuvem e que valorizam agilidade sobre controle total. O contraste não é de superioridade, mas de adequação estratégica ao contexto de negócio e técnico da organização.

A Tabela 1 sintetiza as principais diferenças entre as duas plataformas.

Característica	Jenkins	GitHub Actions
Abordagem	Self-Hosted (Auto-hospedado)	SaaS (Nativo da Nuvem)
Facilidade de Setup	Requer infraestrutura, configuração e manutenção.	Configuração zero para fluxos básicos.
Sintaxe de Pipeline	Scripted Pipeline (Groovy) ou Declarative (Groovy).	Workflow em YAML.
Flexibilidade	Máxima, com controle total sobre o ambiente de build.	Alta, com reusable workflows e composite actions.
Ecossistema	Vasto ecossistema de	Marketplace crescente de

	mais de 1.800 plugins.	milhares de actions.
Escalabilidade	Extremamente escalável, com agents e integração com Kubernetes.	Escalável com runners hospedados pelo GitHub; requer gerenciamento para self-hosted runners em escala.
Custo	Ferramenta gratuita, mas com custos de infraestrutura e operação.	Gratuito para repositórios públicos, com limites de uso para privados.
Segurança	Responsabilidade total do usuário.	Segurança nativa da plataforma com gerenciamento de segredos e ambientes.
Melhor para...	Empresas com infraestrutura legada ou requisitos de compliance específicos, que necessitam de controle total.	Equipes que operam em ambientes nativos da nuvem, que valorizam agilidade e simplicidade.

Tabela 1 – Comparativo Jenkins vs. GitHub Actions
Fonte: elaborado pelo autor (2025)

Estratégias Avançadas de Implantação Contínua

Para a entrega contínua, o estágio de deploy pode se tornar um gargalo ou um ponto de risco se não for gerenciado de forma inteligente. A implantação de um novo software em produção exige um plano cuidadoso para garantir que o serviço permaneça disponível para os usuários e que a transição ocorra de forma segura. Para isso, o mercado adota diversas estratégias avançadas:

- **Implantação Rolling Update (Rolling Updates):** esta estratégia, amplamente utilizada em orquestradores de contêineres como o Kubernetes, envolve a substituição gradual de instâncias da versão antiga pela nova. As novas instâncias são implantadas e, uma vez prontas, as

antigas são removidas. Esse processo continua até que todas as instâncias tenham sido atualizadas. Essa abordagem é eficiente em termos de recursos, pois não requer infraestrutura adicional, mas a reversão pode ser mais lenta.

- **Implantação Blue/Green (Blue/Green Deployment):** o Blue/Green é uma estratégia que visa zero downtime e capacidade de reversão instantânea. O processo envolve a manutenção de dois ambientes idênticos: o ambiente "Blue" (a versão atual em produção) e o "Green" (o novo ambiente com a nova versão). O novo código é implantado e validado no ambiente "Green" enquanto o tráfego de usuários continua no "Blue". Após a validação, o tráfego é instantaneamente redirecionado para o ambiente "Green". O ambiente "Blue" pode ser mantido para uma reversão rápida, caso necessário, ou desativado para economizar custos. Embora ofereça alta segurança e reversão imediata, essa estratégia é mais cara, pois exige a duplicação de recursos.
- **Implantação Canary (Canary Deployment):** a implantação Canary é uma abordagem que minimiza o risco ao expor a nova versão a um pequeno subconjunto de usuários. Uma pequena porcentagem do tráfego é roteada para a nova versão, que é monitorada de perto. Se não houver problemas, o tráfego para a nova versão é gradualmente aumentado até que 100% dos usuários a estejam utilizando. Essa estratégia é ideal para testar novas funcionalidades em um ambiente de produção com risco mínimo e é comumente utilizada em arquiteturas de microsserviços.

GitOps: A Próxima Evolução da Entrega Contínua

Uma das tendências mais importantes na automação de CI/CD é a ascensão da GitOps, uma abordagem de Continuous Delivery que utiliza o Git como a "única fonte de verdade" para a infraestrutura e a configuração de aplicações. Enquanto o CI/CD tradicional se baseia em uma abordagem imperativa (em que o pipeline é instruído a "fazer" algo, como rodar um comando para implantar), o GitOps é **declarativo**. Em um fluxo GitOps, o estado desejado do ambiente (por exemplo, um cluster Kubernetes) é explicitamente definido em arquivos no repositório Git.

O fluxo de trabalho do GitOps é fundamentalmente diferente:

1. A pessoa desenvolvedora faz uma alteração no código ou na configuração e a insere no repositório Git, o que serve como a "fonte de verdade".
2. Um processo automatizado — um **GitOps operator** como o Flux ou o Argo CD — detecta a mudança no repositório.
3. O operador compara o estado real do ambiente com o estado desejado, conforme declarado no Git.
4. Se houver uma discrepância, o operador automaticamente **reconcilia** o ambiente para que ele corresponda ao estado do repositório, garantindo consistência e eliminando a intervenção manual.

As ferramentas de GitOps como o Argo CD e o Flux, projetos hospedados pela CNCF, se tornaram a referência de mercado. O **Argo CD** é conhecido por sua interface gráfica (UI) rica, que simplifica a gestão e o monitoramento de múltiplos clusters, ideal para grandes empresas. Já o **Flux** adota uma abordagem mais minimalista, focada na linha de comando (CLI), o que o torna uma escolha preferida para ambientes com restrições de rede (air-gapped) e para quem busca uma ferramenta mais leve e extensível. Ambos os projetos suportam as estratégias de implantação mais avançadas (Canary e Blue/Green) e são pilares na automação da entrega em ambientes nativos da nuvem e com arquitetura de microsserviços.

Observabilidade em Pipelines: Mais que Logs, Visão Holística

Para manter a saúde e a eficiência de um pipeline de CI/CD, especialmente em ambientes complexos, a **observabilidade** é um pilar essencial. Observabilidade é a capacidade de entender o estado interno de um sistema a partir de seus dados externos, como logs, métricas e rastros (traces). Aplicada ao CI/CD, a observabilidade oferece uma visão aprofundada de todo o processo de entrega de software, desde o check-in do código até a implantação em produção.

Diferentemente do monitoramento tradicional, que se concentra em saber se algo está funcionando, a observabilidade se concentra em entender o porquê de um problema. Ela permite que as equipes de engenharia respondam a perguntas complexas como:

- Por que um build está demorando mais do que o normal?
- Quais testes estão falhando de forma inconsistente (flaky tests)?

- Quais mudanças no código estão gerando gargalos de desempenho?

A observabilidade ajuda a identificar proativamente os pontos de falha e os gargalos de desempenho antes que eles se tornem um problema maior, o que economiza tempo e recursos. Ao analisar dados históricos, as equipes podem tomar decisões mais informadas sobre a alocação de recursos, a adoção de novas ferramentas e a otimização de fluxos de trabalho. A visibilidade e a transparência que a observabilidade proporciona também promovem a colaboração e a responsabilização, permitindo que as equipes resolvam a causa-raiz dos problemas, e não apenas os sintomas.

MERCADO, CASES E TENDÊNCIAS

O mercado de CI/CD está experimentando um crescimento robusto, impulsionado pela crescente adoção de práticas DevOps e arquiteturas nativas da nuvem. As soluções baseadas em nuvem estão superando as soluções on-premises, impulsionadas pela escalabilidade e benefícios de custo.

A aplicação de pipelines de CI/CD em escala global é demonstrada por casos de sucesso que servem como modelo para a transformação digital. O caso da **Netflix** é um exemplo paradigmático de uma empresa que utilizou o CI/CD como um motor de reinvenção do negócio. Ao migrar sua infraestrutura para a Amazon Web Services (AWS), a Netflix pôde escalar globalmente, oferecer alta disponibilidade e inovar rapidamente, transformando-se de um serviço de aluguel de DVDs em um dos maiores produtores de conteúdo global. A automação de seu pipeline, que inclui ferramentas como o Jenkins para integração e o Spinnaker para entrega contínua, foi crucial para gerenciar uma infraestrutura massiva de microsserviços.

A história da **Amazon** ilustra como uma necessidade interna pode se tornar um novo modelo de negócio revolucionário. Para suportar suas operações de e-commerce, a Amazon construiu uma infraestrutura de TI robusta e escalável. Ao perceber que essa capacidade ociosa poderia ser oferecida como um serviço para outras empresas, a Amazon se tornou um dos primeiros provedores de computação em nuvem, criando um mercado inteiramente novo. Hoje, a AWS oferece um conjunto de ferramentas de CI/CD (CodePipeline, CodeBuild, CodeDeploy) que automatizam e aceleram a entrega de software para terceiros.

Esses casos de sucesso demonstram que o CI/CD vai além de uma prática técnica; é um pilar estratégico que pode destravar novos modelos de negócio e proporcionar uma vantagem competitiva inigualável. A excelência em engenharia de software e automação se traduz diretamente em resultados de negócio e liderança de mercado.

O futuro dos pipelines de CI/CD será moldado por tendências tecnológicas emergentes, com a Inteligência Artificial (IA) no centro da inovação. A IA já está sendo utilizada para otimizar workflows, prever falhas de build e alocar recursos de forma dinâmica. Além disso, a IA generativa está sendo usada para automatizar

testes e gerar código. A segurança, que historicamente era um obstáculo, está sendo integrada de forma proativa ao pipeline de CI/CD, uma prática conhecida como **DevSecOps**. A segurança "à esquerda" (shift-left testing), em que as verificações de vulnerabilidades ocorrem no início do ciclo, garante que as vulnerabilidades sejam resolvidas antes que cheguem à produção.

Outra tendência emergente, destacada no Relatório State of DevOps de 2024 da Puppet Labs, é a ascensão da **engenharia de plataformas (Platform Engineering)**. Esta abordagem, vista como a próxima evolução do DevOps, concentra-se na construção de plataformas internas de autoatendimento (self-service) que padronizam e automatizam o ciclo de vida do software. O objetivo é capacitar as equipes de desenvolvimento, liberando-as da complexidade da infraestrutura e dos toolchains para que possam focar na entrega de valor.

A engenharia de plataformas e a integração da IA sinalizam que o futuro do CI/CD vai além da simples automação de tarefas. Trata-se da criação de produtos internos que democratizam e institucionalizam as melhores práticas de entrega de software, transformando o DevOps de uma filosofia para uma função de engenharia estratégica que impulsiona a produtividade em escala.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, o pipeline de CI/CD foi desmistificado, sendo apresentado como a espinha dorsal do desenvolvimento de software moderno. O texto explorou a profunda transformação na entrega de software, desde a definição de Integração Contínua e as duas abordagens de Implantação Contínua, até a sua relação intrínseca com a cultura DevOps.

Foi examinada a anatomia de um pipeline, com seus estágios (Build, Test, Deploy) e componentes arquiteturais (Workflows, Jobs, Steps, Actions), enfatizando a importância da automação e da prática de Pipeline as Code. As características e os casos de uso de duas das ferramentas mais proeminentes no mercado, Jenkins e GitHub Actions, foram analisados, destacando o dilema entre controle e conveniência que permeia as decisões de arquitetura.

Finalmente, foram apresentados dados de mercado e casos de sucesso globais da Netflix e da Amazon, demonstrando como o CI/CD se tornou um pilar estratégico para a inovação e o crescimento de negócios. A aula também discutiu as tendências futuras, como a integração de Inteligência Artificial para otimização de pipelines e a ascensão da engenharia de plataformas, preparando o aluno e aluna para os desafios e oportunidades do mercado de tecnologia em 2025 e além.

REFERÊNCIAS

CATOPS, W. **A Complete Guide to Creating GitHub Actions Pipeline with YAML Templates**. 2024. Disponível em: <https://blog.devops.dev/a-complete-guide-to-creating-github-actions-pipeline-with-yaml-templates-c57f2dbc2d0c>. Acesso em: 07 out. 2025.

FORTINET. **What Is Shift Left Security?**. 2025. Disponível em: <https://www.fortinet.com/resources/cyberglossary/shift-left-security>. Acesso em: 07 out. 2025.

MARGAM, V. **Efficient CI/CD with Jenkins: Exploring Master-Slave Architecture and Agents**. 2023. Disponível em: <https://varunmargam.hashnode.dev/efficient-cicd-with-jenkins-exploring-master-slave-architecture-and-agents>. Acesso em: 07 out. 2025.

MARIA. **Jenkins vs GitHub Actions: CI/CD Showdown for 2025**. 2025. Disponível em: <https://everhour.com/blog/jenkins-vs-github-actions/>. Acesso em: 07 out. 2025.

MINICK, E. **What is the GitOps workflow**. 2024. Disponível em: <https://www.harness.io/blog/what-is-the-gitops-workflow>. Acesso em: 07 out. 2025.

MIRAMAR, H. **10 Real-World CI CD Pipeline Examples That Drive DevOps Success**. 2025. Disponível em: <https://blog.mergify.com/real-world-ci-cd-pipeline-examples-devops-success/>. Acesso em: 07 out. 2025.

NETFLIX. **Netflix's Coding Revolution: The CI-CD Story Unveiled**. 2025. Disponível em: <https://44243536.fs1.hubspotusercontent-na1.net/hubfs/44243536/Blog%20CTA%20Documents/Netflixs%20Coding%20Revolution%20The%20CI-CD%20Story%20Unveiled.pdf>. Acesso em: 07 out. 2025.

PERFORCE. **Puppet's 2024 State of DevOps Report Reveals Security is Strengthened by Platform Engineering**. 2024. Disponível em: <https://www.perforce.com/press-releases/2024-state-devops-report>. Acesso em: 07 out. 2025.

RED HAT. **O que é um pipeline de CI/CD?**. 2022. Disponível em: <https://www.redhat.com/pt-br/topics/devops/what-cicd-pipeline>. Acesso em: 07 out. 2025.

TROTTA, F. **Mastering AI-Enhanced CI/CD Pipelines for Optimal Software Delivery**. 2025. Disponível em: <https://zencoder.ai/blog/building-ai-enhanced-ci-cd-pipelines-for-enterprise-applications>. Acesso em: 07 out. 2025.

PALAVRAS-CHAVE

CI/CD. DevOps. Pipeline as Code. GitHub Actions. Jenkins. Automação. Cloud-Native. DevSecOps. Inteligência Artificial. Platform Engineering.

EMSE

The background is a dark, textured surface with a network of thin, glowing teal lines. These lines connect various nodes, some of which are represented by small teal squares or rectangles. The overall effect is a digital or technological aesthetic, reminiscent of a data network or a futuristic interface.

POSTECH